

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

NAME: M Manoj Reddy

REG NO: 192372282

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 3

Duration : 1 Hour

Total Marks: 30

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

*I, **M Manoj Reddy (192372282)**, certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: M Manoj Reddy

Question 1: Doctor Shift Scheduling using Backtracking Search

Objective

To assign three doctors (D1, D2, D3) to three shifts (Morning, Afternoon, Night) such that all given scheduling constraints are satisfied, by applying the Backtracking Search algorithm to a Constraint Satisfaction Problem (CSP), and to demonstrate every intermediate assignment and constraint-check step.

Problem Statement

A hospital must assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night), one doctor per shift, subject to:

- i D1 cannot work the Night shift.
- ii D2 must work before D3, using the ordering Morning < Afternoon < Night.
- iii Only one doctor may be assigned per shift (all-different constraint).
- iv D3 cannot work the Morning shift.
- v Every doctor must be assigned exactly one shift.

Solution Approach

This is modelled as a CSP with variables {D1, D2, D3}, domain {Morning, Afternoon, Night} for each variable, and the five constraints listed above. Backtracking Search assigns variables one at a time in the order D1 → D2 → D3. After every partial assignment, all applicable constraints are checked immediately (forward constraint checking). If a value violates a constraint, that branch is pruned and the next value in the domain is tried; if no value works, the algorithm backtracks to the previous variable.

Step-by-step trace:

- 1 **D1 = Morning** → check constraint (i): D1 ≠ Night → satisfied. Move to D2.
- 2 **D2 = Morning** → check all-different: D1 = D2 = Morning → VIOLATED. Backtrack.
- 3 **D2 = Afternoon** → all-different satisfied (Morning, Afternoon distinct). Move to D3.
- 4 **D3 = Morning** → check constraint (iv): D3 ≠ Morning → VIOLATED. Backtrack.
- 5 **D3 = Afternoon** → check all-different: D2 = D3 = Afternoon → VIOLATED. Backtrack.
- 6 **D3 = Night** → check all-different: satisfied. Check constraint (ii): order(D2=Afternoon)=1 < order(D3=Night)=2 → satisfied. All 5 constraints hold and all variables assigned → **GOAL STATE REACHED**.

Final Valid Schedule: D1 → Morning, D2 → Afternoon, D3 → Night.

Doctor	Assigned Shift
D1	Morning
D2	Afternoon
D3	Night

Code (Python implementation of Backtracking Search)

```
shifts_order = {"Morning": 0, "Afternoon": 1, "Night": 2}
doctors = ["D1", "D2", "D3"]
```

```

domain = ["Morning", "Afternoon", "Night"]

def constraints_ok(assignment):
    if assignment.get("D1") == "Night":          # (i)
        return False
    if assignment.get("D3") == "Morning":         # (iv)
        return False
    if "D2" in assignment and "D3" in assignment: # (ii)
        if shifts_order[assignment["D2"]] >= shifts_order[assignment["D3"]]:
            return False
    values = list(assignment.values())             # (iii) all-different
    if len(values) != len(set(values)):
        return False
    return True

solution = None

def backtrack(assignment):
    global solution
    if len(assignment) == len(doctors):          # (v) all assigned
        solution = dict(assignment)
        return True
    var = doctors[len(assignment)]
    for value in domain:
        assignment[var] = value
        if constraints_ok(assignment):
            if backtrack(assignment):
                return True
        del assignment[var]
    return False

backtrack({})
print(solution)

```

Output

```

Try D1 = Morning -> passes constraint check {'D1': 'Morning'}
Try D2 = Morning -> FAILS constraint check, backtrack
Try D2 = Afternoon -> passes constraint check {'D1': 'Morning', 'D2': 'Afternoon'}
Try D3 = Morning -> FAILS constraint check, backtrack
Try D3 = Afternoon -> FAILS constraint check, backtrack
Try D3 = Night -> passes constraint check {'D1': 'Morning', 'D2': 'Afternoon', 'D3': 'Night'}
All variables assigned -> VALID (goal reached)

Final Valid Schedule: {'D1': 'Morning', 'D2': 'Afternoon', 'D3': 'Night'}

```

Discussion

Backtracking Search is well suited to this problem because the search space is small (only $3^3 = 27$ raw combinations) and the constraints are simple binary/global relations that can be checked incrementally. Early constraint checking (as soon as a variable is assigned) prunes invalid branches quickly, which is why only 6 candidate assignments were tried before the valid schedule was found instead of exploring all 27 combinations. This illustrates the efficiency gained from constraint propagation compared to blind generate-and-test search.

Conclusion

The Backtracking Search algorithm successfully found the unique valid schedule satisfying all five constraints: $D1 \rightarrow \text{Morning}$, $D2 \rightarrow \text{Afternoon}$, $D3 \rightarrow \text{Night}$. This confirms that CSP-based backtracking is an effective and systematic technique for constraint-based scheduling problems.

Question 2: Robot Path Planning in a 5x5 Grid using Search with Priority Queue

Objective

To find the optimal (least-cost) path for a robot moving from a Start cell (S) to a Goal cell (G) in a 5x5 grid containing obstacles, using a priority-queue-based search guided by the Manhattan Distance heuristic, and to show the step-by-step node expansion and the resulting optimal path and cost.

Problem Statement

A robot operates on a 5x5 grid and must travel from Start (S) to Goal (G) under the following constraints:

- i Allowed moves: Up, Down, Left, Right.
- ii Cost of each move = 1.
- iii The robot cannot pass through obstacle cells.
- iv The Manhattan Distance heuristic, $h(n) = |x_{\text{start}} - x_{\text{goal}}| + |y_{\text{start}} - y_{\text{goal}}|$, guides node selection.

For this trace, S is placed at grid cell (0,0) (top-left) and G at cell (4,4) (bottom-right), with obstacle cells at (0,3), (1,1), (1,3), (2,1), (3,1), (3,2), (3,3) (0-indexed as row, column):

S	.	.	X	.
.	X	.	X	.
.	X	.	.	.
.	X	X	X	.
.	.	.	.	G

S = Start, G = Goal, dark cells (X) = obstacles.

Solution Approach

The search maintains a priority queue (min-heap) of frontier nodes ordered by $f(n) = g(n) + h(n)$, where $g(n)$ is the actual path cost from S to node n, and $h(n)$ is the Manhattan Distance from n to G. At each step, the node with the lowest $f(n)$ is popped and expanded; its unvisited neighbours (Up/Down/Left/Right, excluding obstacles and out-of-grid cells) are generated, their f-values computed, and pushed back into the priority queue. This continues until G is popped from the queue, which guarantees the optimal path because the Manhattan Distance heuristic is admissible (never overestimates the true remaining cost) for grid movement with unit step cost.

Code (Python implementation using heapq priority queue)

```
import heapq

grid = [
    [0,0,0,1,0],
    [0,1,0,1,0],
```

```

    [0,1,0,0,0],
    [0,1,1,1,0],
    [0,0,0,0,0],
] # 0 = free, 1 = obstacle
START, GOAL = (0,0), (4,4)

def neighbors(pos):
    r, c = pos
    moves = [("Up", (r-1,c)), ("Down", (r+1,c)), ("Left", (r,c-1)), ("Right", (r,c+1))]
    return [(n,(nr,nc)) for n,(nr,nc) in moves
            if 0<=nr<5 and 0<=nc<5 and grid[nr][nc]==0]

def manhattan(a,b):
    return abs(a[0]-b[0]) + abs(a[1]-b[1])

open_list = [(manhattan(START,GOAL), 0, 0, START, [START])]
visited, g_score, counter = set(), {START:0}, 0

while open_list:
    f, g, _, current, path = heapq.heappop(open_list)
    if current in visited:
        continue
    visited.add(current)
    if current == GOAL:
        break
    for name, npos in neighbors(current):
        if npos in visited:
            continue
        tg = g + 1
        if npos not in g_score or tg < g_score[npos]:
            g_score[npos] = tg
            h = manhattan(npos, GOAL)
            counter += 1
            heapq.heappush(open_list, (tg+h, tg, counter, npos, path+[npos]))

print("Optimal path:", path)
print("Total cost:", g)

```

Output — Node Expansion and Priority Queue Trace

```

Step 1: Expand (0,0) | g=0, h=8, f=8
    Generate Down -> (1,0) | g=1, h=7, f=8    [pushed to PQ]
    Generate Right -> (0,1) | g=1, h=7, f=8    [pushed to PQ]
Step 2: Expand (1,0) | g=1, h=7, f=8
    Generate Down -> (2,0) | g=2, h=6, f=8    [pushed to PQ]
Step 3: Expand (0,1) | g=1, h=7, f=8
    Generate Right -> (0,2) | g=2, h=6, f=8    [pushed to PQ]
Step 4: Expand (2,0) | g=2, h=6, f=8
    Generate Down -> (3,0) | g=3, h=5, f=8    [pushed to PQ]
Step 5: Expand (0,2) | g=2, h=6, f=8
    Generate Down -> (1,2) | g=3, h=5, f=8    [pushed to PQ]
Step 6: Expand (3,0) | g=3, h=5, f=8
    Generate Down -> (4,0) | g=4, h=4, f=8    [pushed to PQ]
Step 7: Expand (1,2) | g=3, h=5, f=8
    Generate Down -> (2,2) | g=4, h=4, f=8    [pushed to PQ]
Step 8: Expand (4,0) | g=4, h=4, f=8
    Generate Right -> (4,1) | g=5, h=3, f=8    [pushed to PQ]
Step 9: Expand (2,2) | g=4, h=4, f=8
    Generate Right -> (2,3) | g=5, h=3, f=8    [pushed to PQ]
Step 10: Expand (4,1) | g=5, h=3, f=8
    Generate Right -> (4,2) | g=6, h=2, f=8    [pushed to PQ]
Step 11: Expand (2,3) | g=5, h=3, f=8
    Generate Right -> (2,4) | g=6, h=2, f=8    [pushed to PQ]
Step 12: Expand (4,2) | g=6, h=2, f=8
    Generate Right -> (4,3) | g=7, h=1, f=8    [pushed to PQ]
Step 13: Expand (2,4) | g=6, h=2, f=8
    Generate Up -> (1,4) | g=7, h=3, f=10    [pushed to PQ]
    Generate Down -> (3,4) | g=7, h=1, f=8    [pushed to PQ]

```

```
Step 14: Expand (4,3) | g=7, h=1, f=8  
        Generate Right -> (4,4) | g=8, h=0, f=8    [pushed to PQ]  
Step 15: Expand (3,4) | g=7, h=1, f=8  
Step 16: Expand (4,4) | g=8, h=0, f=8    -> GOAL REACHED
```

Optimal Path:

(0,0) -> (1,0) -> (2,0) -> (3,0) -> (4,0) -> (4,1) -> (4,2) -> (4,3) -> (4,4)

Total Optimal Cost: 8 moves

Discussion

The priority queue always expanded the node with the lowest $f(n)$, which kept the search focused along the boundary of the obstacle block rather than exploring irrelevant branches. Because the Manhattan Distance heuristic never overestimates the true cost on a grid with only orthogonal unit moves, the search is admissible and consistent, so the first time G is popped from the queue the path found is guaranteed optimal. Here the path cost (8) exactly equals the straight-line Manhattan distance between S and G, meaning the obstacles did not force any detour beyond the theoretical minimum.

Conclusion

Using a priority-queue-based search with the Manhattan Distance heuristic, the robot's optimal path from S(0,0) to G(4,4) was determined as $(0,0) \rightarrow (1,0) \rightarrow (2,0) \rightarrow (3,0) \rightarrow (4,0) \rightarrow (4,1) \rightarrow (4,2) \rightarrow (4,3) \rightarrow (4,4)$ with a total cost of 8, confirming that heuristic-guided search efficiently finds optimal obstacle-avoiding paths in grid environments.

Question 3: Autonomous Rescue Robot in a Partially Observable Environment

Objective

To design a search-based decision-making approach for an autonomous rescue robot that must navigate from a start position (S) to a survivor location (G) in a partially observable, dynamically-sensed grid environment, while minimising total movement cost and analysing the problem through core AI agent concepts.

Problem Statement

The rescue robot moves in a grid of rooms from S to G under the following constraints:

- i Movement allowed: Up, Down, Left, Right.
- ii Some cells are blocked (obstacles) and cannot be traversed.
- iii The robot has limited sensing — it can only observe adjacent cells (partial observability).
- iv Each move costs 1; entering a risky zone costs an additional 2 (total 3).
- v The robot must minimise total cost while navigating safely.
- vi The robot must update its knowledge dynamically as it senses new information (online search).

a) How the Constraints Affect Decision-Making

- a **Blocked cells:** restrict the action set at each state — the robot must verify a neighbouring cell is free before committing to a move, which prevents an a-priori full path from being planned offline.
- b **Limited sensing (partial observability):** the robot only knows the status of adjacent cells, so its internal map is incomplete; it must explore incrementally and revise its belief state as new cells become visible, rather than relying on a search over a fully known state space.
- c **Variable move cost (risky-zone penalty):** turns the problem into a weighted-path optimisation rather than a simple shortest-hop-count problem — the cheapest path in terms of number of steps may not be the cheapest in terms of total cost if it passes through risky zones.
- d **Dynamic knowledge update:** forces the robot to interleave planning and execution — after every move it re-evaluates its local knowledge and re-plans if the previously assumed path is no longer valid or a cheaper option is revealed.

b) Solution Approach: Online Uniform Cost Search (UCS)

Because the environment is only partially observable, a purely offline search (such as computing the full path with UCS before moving, as would be possible in Question 2) is not directly applicable. Instead, an **Online Search Agent using Uniform Cost Search principles** is used:

- 1 The robot maintains a priority queue of known-reachable frontier cells, ordered by accumulated path cost $g(n)$ (1 for a normal move, 3 for a move into a sensed risky cell).
- 2 At its current position, it senses all adjacent cells to discover which are free, blocked, or risky, and updates its internal knowledge map accordingly.
- 3 It selects and physically moves to the lowest-cost frontier cell that is consistent with its current knowledge (it cannot "peek" at unsensed cells the way an offline search could).

- 4 If a move reveals that an assumed-free cell is actually blocked, the robot updates g-values / removes that edge and re-computes the next best action from its current state — this is the hallmark of online (interleaved plan-and-act) search rather than offline search.
- 5 The process repeats until the robot reaches G, at which point the accumulated g-value is the total realised cost of the rescue path.

c) Application of Core AI Concepts

Concept	Application in this Scenario
Intelligent Agent	The rescue robot is an agent that perceives its environment through local sensors (adjacent-cells)
Rationality	At each decision point the robot chooses the action that, given its current (incomplete) knowledge, minimises the accumulated cost function
Environment Type	The environment is: partially observable (only adjacent cells sensed), dynamic in the robot's knowledge

d) Justification of the Chosen Approach

Online Uniform Cost Search is the most suitable strategy here because it directly matches the two defining constraints of the problem: partial observability and variable move cost. A classical offline algorithm (plain BFS or a fully pre-computed A*) assumes the entire state space — including which cells are blocked or risky — is known in advance, which does not hold here since the robot can only sense adjacent cells. Online UCS naturally interleaves sensing, cost-based decision-making, and movement, allowing the robot to always commit to the cheapest currently-known-safe action and to gracefully re-plan whenever new information (e.g., a hidden obstacle or risky zone) is revealed. This keeps the robot's behaviour both safe (never attempts a known-blocked cell) and cost-rational (always minimises the accumulated cost function), which is exactly what is required for disaster-zone navigation under uncertainty.

Discussion

Unlike Questions 1 and 2, where the full problem structure (all constraints, or the entire grid and obstacle layout) was known beforehand, Question 3 introduces uncertainty through partial observability. This shifts the appropriate technique from a search that can be solved completely offline to an online agent architecture where perception and action are tightly coupled. The risky-zone cost penalty also means the robot must reason not just about reachability but about a genuine cost trade-off, favouring a longer but cheaper detour over a short but risky direct route when the numbers justify it.

Conclusion

For the autonomous rescue robot operating under partial observability with variable movement costs, an Online Uniform Cost Search agent is the most appropriate solution: it satisfies rationality by always selecting the least-cost known-safe action, respects the partially observable and dynamic nature of the environment by continuously updating its knowledge, and safely guides the robot from S to G while minimising total rescue-mission cost.